

Chapter 9

Consistency and Consensus

General-purpose abstractions for agreement — from linearizability to Paxos/Raft

Source: [claude-skills / ch09-consistency-and-consensus.md](#) · Illustrated · Interview-ready

Martin Kleppmann · Part II: Distributed Data

Table of Contents

1. Core Idea
2. Framework: Linearizability
3. Framework: Two-Phase Commit (2PC)
4. Linearizability vs. Serializability
5. CAP Theorem & Kleppmann's Critique
6. Causal Consistency
7. Lamport Timestamps vs. Version Vectors
8. Total Order Broadcast
9. FLP Impossibility Result
10. Fault-Tolerant Consensus: Raft, Paxos & Friends
11. Epochs and Quorums
12. ZooKeeper & etcd
13. Mental Models
14. Anti-Patterns
15. Worked Example: 2PC Coordinator Crash
16. Problems Each Mechanism Solves
17. Connects To (Cross-Chapter)
18. Key Takeaways
19. Junior SWE Checkpoints

1. Core Idea

The best way to build fault-tolerant distributed systems is to find **general-purpose abstractions with useful guarantees**, implement them once, and let applications rely on them — the same move transactions made for single-node faults.

Key abstraction

Consensus — getting all nodes to agree on something.

The ladder

Linearizability → causal ordering → total order
broadcast → consensus.

Equivalence

The top rungs are all *the same problem in disguise*.

Climb the ladder: start with the strongest single-object guarantee (linearizability), discover cheaper middle ground (causal consistency), then realize total order broadcast and consensus are equivalent — solve one, you've solved leader election, atomic commit, locks, uniqueness constraints, and linearizable compare-and-set.

The Consistency & Consensus Ladder

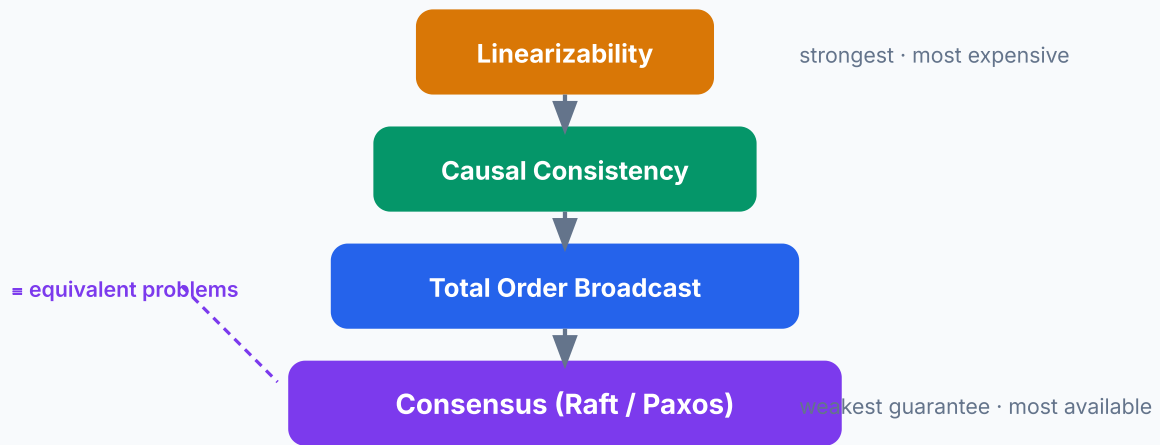


Figure 1.1 — Each rung trades cost for weaker guarantees; top three are reducible to consensus.

2. Framework: Linearizability

Also: atomic consistency, strong consistency, immediate consistency. The **strongest single-object guarantee**.

When	Locking/leader election, uniqueness constraints (usernames, seat booking), cross-channel timing dependencies (message queue racing a file store).
How	Make the system appear as if there is only a single copy of the data and all operations are atomic. Once any client reads a new value, all subsequent reads (by any client) must return it — a recency guarantee .
Why it works	Every operation takes effect atomically at some point between its start and end → one timeline all clients agree on.
Failure mode	Expensive. Single-leader is only <i>potentially</i> linearizable; multi-leader is not; Dynamo-style quorums ($w+r>n$) are not — concurrent readers can still see old and new values. Attiya & Welch: linearizable response times are inevitably high in variable-delay networks. Even multicore RAM isn't linearizable.

Linearizability: single timeline all clients see

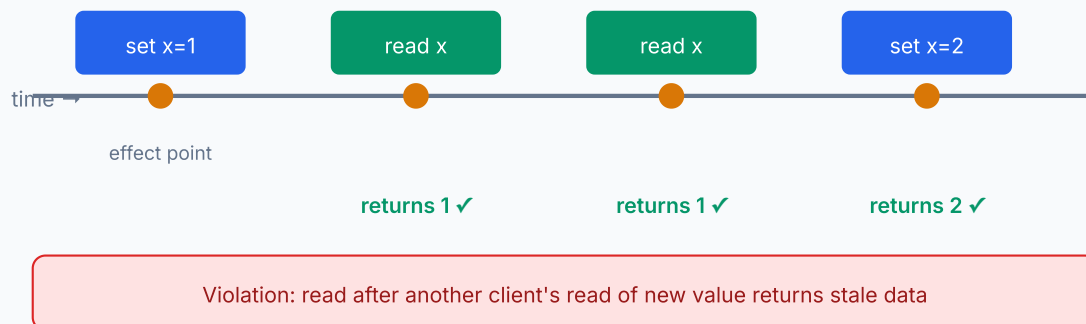


Figure 2.1 — Operations appear atomic at a point on a single global timeline.

What is (and isn't) linearizable

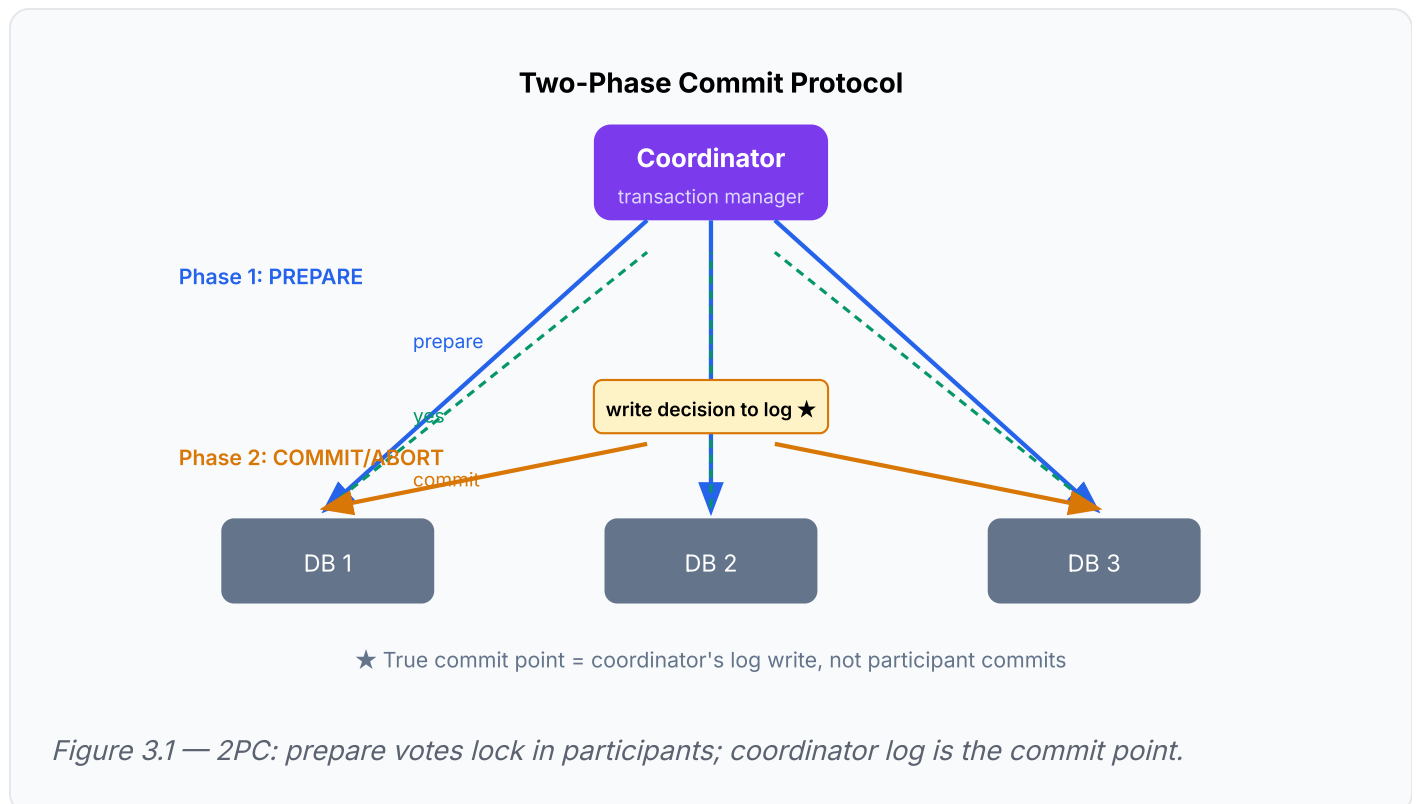
System	Linearizable?	Why
Single-leader + sync follower reads	Potentially ✓	Leader serializes writes; sync replication + leader reads
Single-leader + follower reads	✗	Lagging followers return stale values
Multi-leader	✗	Concurrent writes on different leaders
Leaderless quorum ($w+r>n$)	✗	Concurrent readers can see old and new values simultaneously
Consensus-backed (Raft/ZK)	✓	Majority quorums + single leader per epoch

Real systems: etcd, ZooKeeper (linearizable reads via sync), Spanner (TrueTime-bounded staleness), FoundationDB (strict serializability). Most "strong consistency" claims need scrutiny — ask *which operations* and *under what failure model*.

3. Framework: Two-Phase Commit (2PC)

Also: XA transactions, distributed atomic commit. Used for **atomic commit across multiple nodes/partitions** or heterogeneous systems.

When	Multiple services/databases must commit or abort together despite no shared transaction manager.
How	A coordinator (transaction manager) sends <i>prepare</i> to all participants. Each replies yes only if it can definitely commit (writes durably, surrenders abort right). If all yes → coordinator writes decision to its log (the true commit point), then sends commit/abort.
Why it works	Participants can't unilaterally decide; coordinator's log is the single source of truth for the outcome.
Failure mode	Blocking. Coordinator crash after prepare leaves participants <i>in doubt</i> — they hold locks until coordinator recovers. Safety but not termination . Not fault-tolerant consensus.



2PC $\neq$ consensus. It provides *safety* (atomicity) but not *termination* (liveness). A crashed coordinator blocks everything. Fault-tolerant consensus (Raft/Paxos) adds majority quorums and epochs to guarantee progress.

4. Linearizability vs. Serializability

Interview-sharp distinction. These sound similar but operate at different layers.

	Linearizability	Serializability
Scope	Single object ("register")	Multi-object <i>transactions</i>
Guarantee	Recency — reads see latest write	Transactions behave as if executed in <i>some</i> serial order
Transactions?	No — individual reads/writes	Yes — isolation property
Cross-object ordering	None	Yes — serial order across all objects in txn
Example systems	ZooKeeper CAS, etcd	PostgreSQL SERIALIZABLE, FoundationDB

Both together = strict serializability (strong-1SR). 2PL-based serializability is typically linearizable. **SSI is serializable but NOT linearizable** — it reads from a consistent snapshot, which is stale by design (Ch 7).

Venn: where guarantees overlap

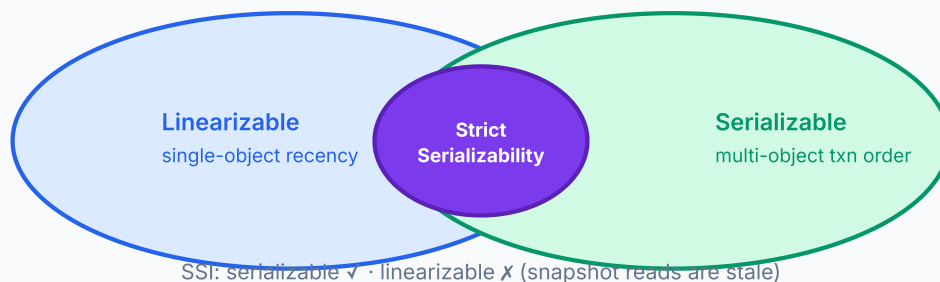


Figure 4.1 — Strict serializability = both guarantees combined.

5. CAP Theorem & Kleppmann's Critique

C — Consistent

Linearizability: all nodes see the same data at the same time.

A — Available

Every request gets a response (not an error), even if it's stale.

P — Partition-tolerant

System continues despite network partitions between nodes.

CAP claim: during a network partition you choose linearizability *or* availability — not both.

CAP during a network partition

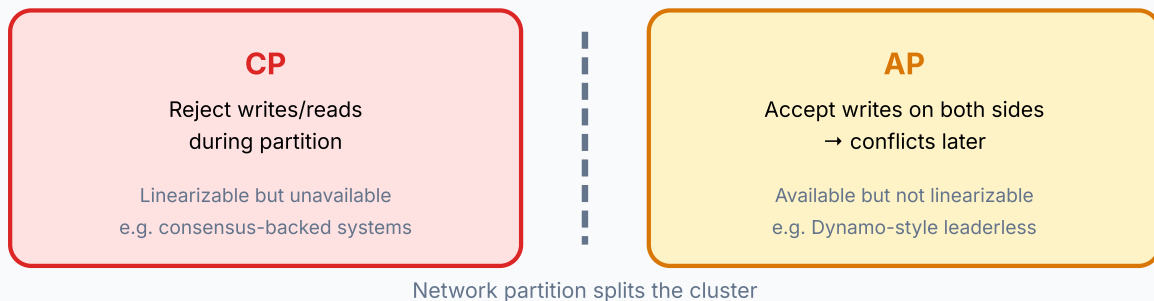


Figure 5.1 — CAP forces a choice only during partitions.

Kleppmann's critique — CAP is mostly of historical interest:

- Considers **one** consistency model (linearizability) and **one** fault (partitions)
- Says nothing about delays, crashes, or trade-offs in *normal operation*
- **CP/AP labels mislead** — many real systems are neither cleanly CP nor AP
- Avoid CAP for precise reasoning; use specific guarantees instead

6. Causal Consistency

The **strongest consistency model** that doesn't lose availability under partitions.

Property	Detail
Partial order	Causality gives a partial order — concurrent ops are <i>incomparable</i>
vs linearizability	Linearizability implies causality (total order). Causal is weaker and cheaper.
Practical sweet spot	Many apps that "need" linearizability actually only need causal consistency

Causal vs Linear (total) order

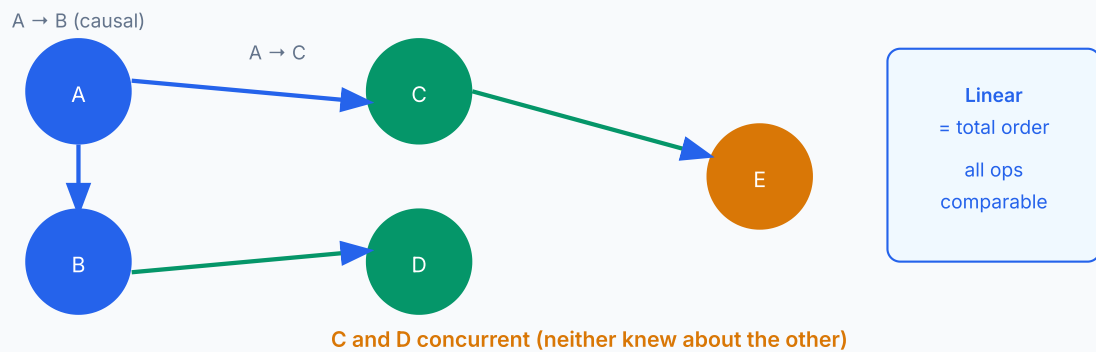


Figure 6.1 — Causal order preserves happens-before; concurrent ops need no ordering.

📱 **Social media:** You post a comment, then your friend replies. Causal consistency ensures everyone sees your post before the reply. But two strangers posting simultaneously don't need a global order — that's overkill (and expensive).

7. Lamport Timestamps vs. Version Vectors

Two different tools for ordering — don't confuse them.

	Lamport Timestamp	Version Vector
Format	(counter, node ID)	One counter per replica per key
Ordering	Total order <i>consistent with</i> causality	Detects concurrent vs causally ordered
Concurrency detection	× Cannot tell if two ops are concurrent	✓ Explicitly identifies concurrent writes
Size	Compact (2 numbers)	Grows with replica count
Uniqueness constraints	× Winner only known after collecting all ops	✓ Can detect conflicts at write time

Same scenario, two approaches

Lamport Timestamps

Node A: (3, A) → (4, A)

Node B: (3, B) → (5, B)

Total order: (3,A) < (3,B) < (4,A) < (5,B)

But (3,A) and (3,B) might be concurrent!

Lamport can't tell — picks arbitrary order

Good for: logging, debugging, rough ordering

Version Vectors

Write on A: vv = [A:1, B:0]

Write on B: vv = [A:0, B:1]

Neither dominates →

concurrent Server keeps both as siblings App resolves or merges Good for: conflict detection (Ch 5 leaderless)

Figure 7.1 — Lamport gives total order; version vectors detect concurrency.

Don't use Lamport clocks for uniqueness constraints. Two clients can both "claim" a username with concurrent Lamport timestamps — the winner is only final after collecting all operations. Use consensus (compare-and-set on a log) instead.

8. Total Order Broadcast

Messages delivered **reliably, in the same order, to all nodes** — the order is fixed at delivery time (stronger than timestamp ordering).

Property	Detail
Reliable delivery	Every message delivered to every node (no loss)
Total order	All nodes see messages in the identical sequence
Same as	A replication log (Ch 5 single-leader)
Equivalent to	Consensus — linearizable CAS $\leftrightarrow$ total order broadcast

Total Order Broadcast = Replication Log

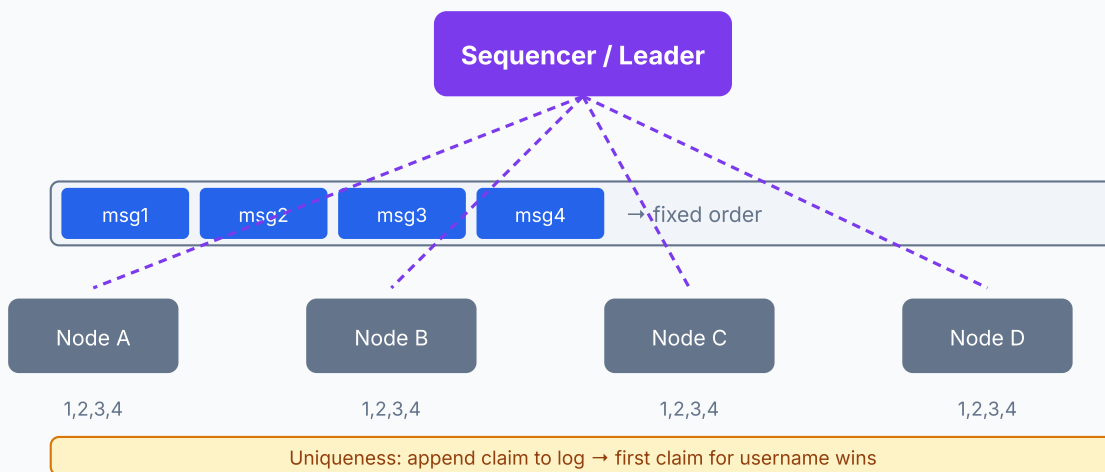


Figure 8.1 — Total order broadcast: same sequence everywhere. Built CAS on it for uniqueness.

Consensus reductions: linearizable compare-and-set, atomic commit, total order broadcast, locks/leases, uniqueness constraints, and leader election are all *reducible to consensus*. Solve one, you've solved them all.

9. FLP Impossibility Result

Fischer, Lynch, Paterson (1985): consensus is impossible with a deterministic algorithm in the asynchronous model if a node may crash.

Assumption	Implication
Asynchronous network	No bound on message delay — can't distinguish slow from dead
Deterministic algorithm	Same inputs → same outputs, always
One crash allowed	Even a single faulty node makes consensus impossible

FLP: why consensus seems impossible — and why we build it anyway

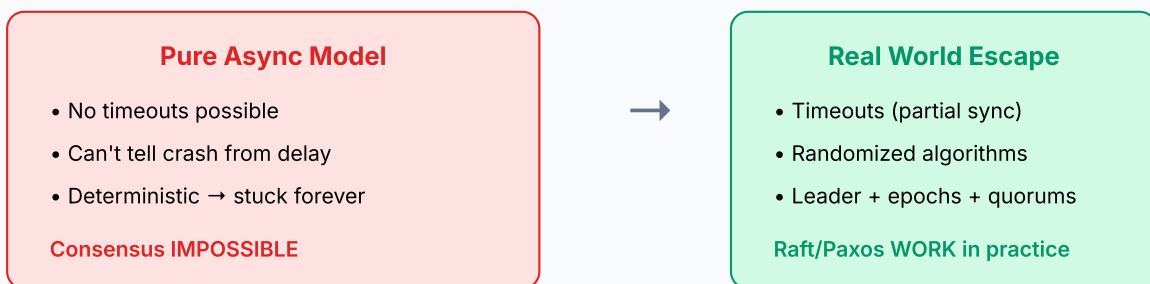


Figure 9.1 — FLP is great theory, limited practical restriction. Timeouts rescue us.

Great theoretical importance, limited practical restriction. FLP tells us pure async consensus can't exist — but adding timeouts or randomness (Raft election timeouts, Paxos randomized proposers) makes it solvable. Every production consensus system "cheats" on the async model.

10. Fault-Tolerant Consensus: Raft, Paxos & Friends

What 2PC lacks: **termination** — the liveness property that guarantees progress even when nodes fail.

Consensus properties

Property	Meaning
Uniform agreement	All non-faulty nodes decide the same value
Integrity	No node decides twice, and decided value was proposed
Validity	If everyone proposes the same value, that's what's decided
Termination	Every non-faulty node eventually decides (2PC lacks this!)

Algorithms

Algorithm	Notes	Used by
Viewstamped Replication (VSR)	View numbers = epochs	Research predecessor
Paxos	Single-value; Multi-Paxos for sequences	Chubby (Google), historical
Raft	Designed for understandability; leader + log replication	etcd, CockroachDB, TiKV, Consul
Zab	Similar to Raft; total order broadcast	ZooKeeper

Raft: Leader Election + Log Replication

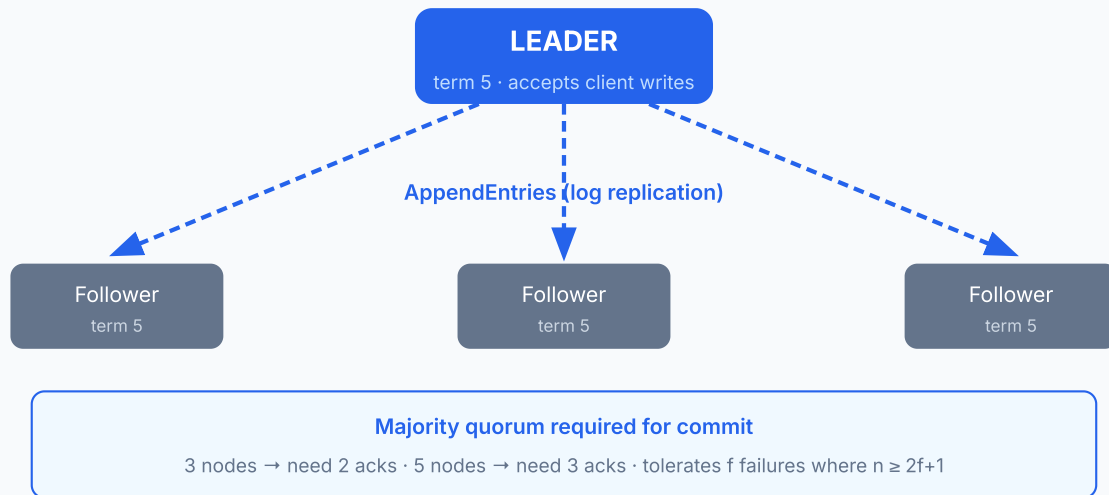


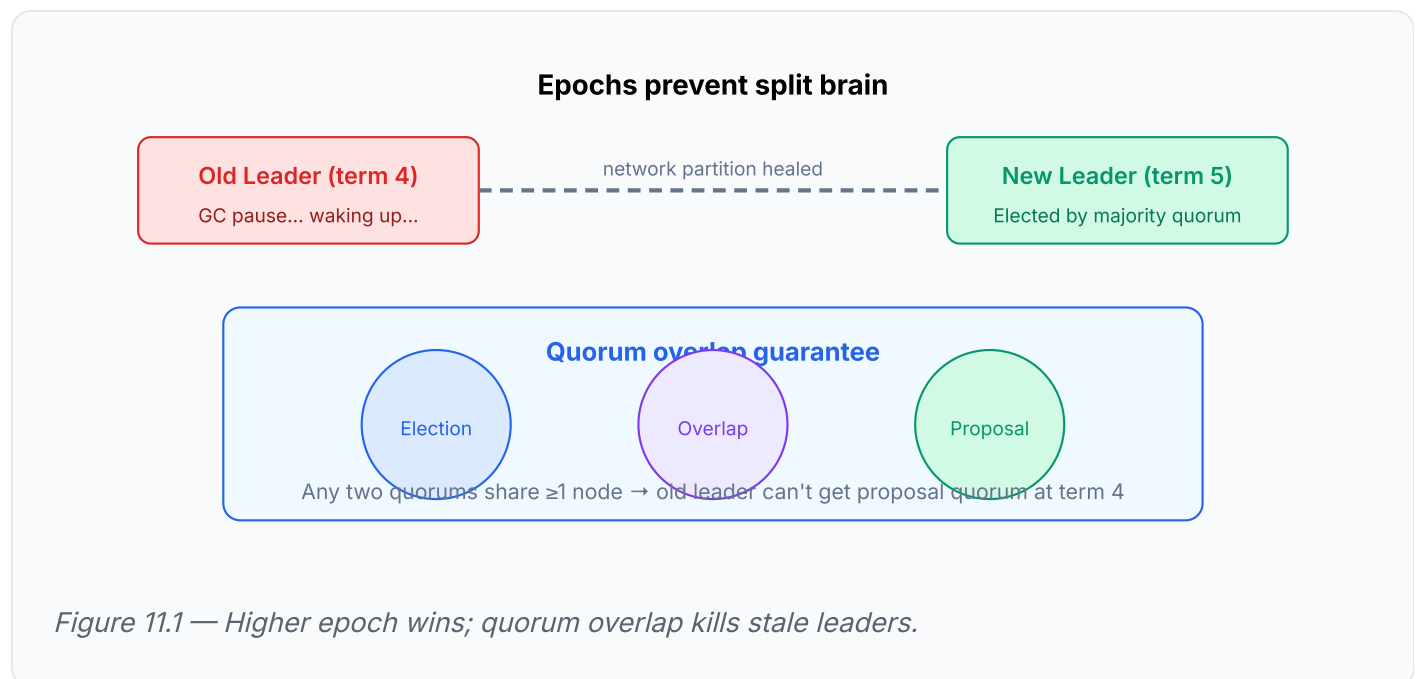
Figure 10.1 — Raft: one leader per term, majority quorum for every decision.

Most consensus algorithms decide a *sequence of values* — they implement total order broadcast directly. Don't implement consensus yourself: use ZooKeeper, etcd, or a database with built-in Raft.

11. Epochs and Quorums

Resolves the chicken-and-egg: *consensus uses a leader, but electing a leader needs consensus.*

Concept	Detail
Epoch number	Monotonically increasing per leader election (ballot in Paxos, view in VSR, term in Raft)
Leader confirmation	Leader must get quorum to confirm no higher-epoch leader exists before deciding
Two quorums	Election quorum + proposal quorum — overlapping quorums prevent split brain



Fencing tokens: A lock alone doesn't guarantee uniqueness — a paused (GC) old leader can wake and corrupt data. Use **monotonic fencing tokens** (ZooKeeper zxid) so downstream services reject stale leaders' requests.

12. ZooKeeper & etcd

Consensus outsourced. Don't roll your own — use a coordination service.

Feature	What it provides	Use case
Linearizable CAS	Compare-and-set with monotonic zxid fencing tokens	Locks, leases, uniqueness
Ephemeral nodes	Auto-deleted when session expires	Leader election, liveness detection
Change notifications	Watch for data changes (no polling)	Service discovery, config updates
Sequential nodes	Monotonic sequence numbers	Queue, lock ordering

ZooKeeper/etcd as coordination layer

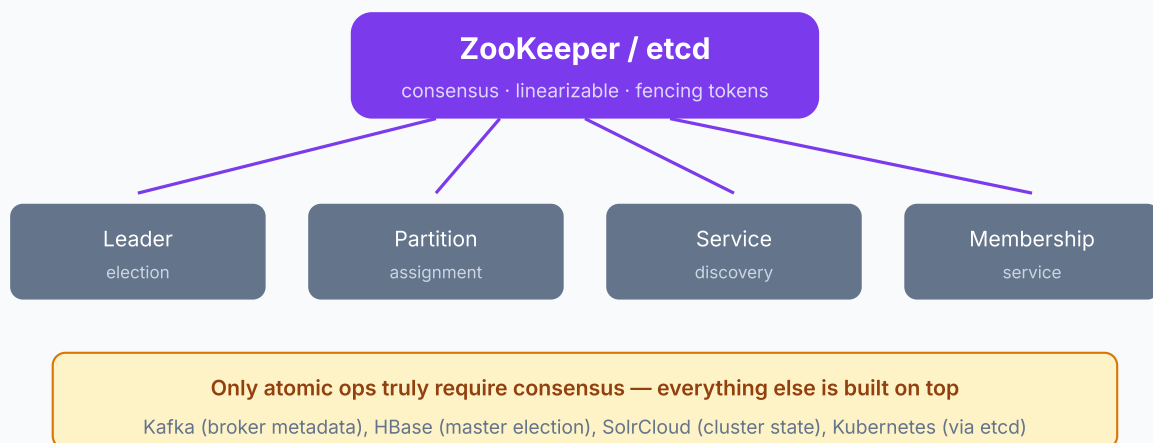


Figure 12.1 — ZooKeeper sits under half the infrastructure you use.

Exactly-once message processing: atomically commit message acknowledgment + database writes (heterogeneous transaction via 2PC or idempotent consumer) → message is *effectively* processed exactly once, even with retries. Returns in Ch 11 stream processing.

13. Mental Models

1. Single-copy illusion

Linearizability = "behave as though there is only one copy of the data and all operations are atomic." Everything about its cost follows from maintaining that illusion across replicas.

2. Consensus reductions

Linearizable CAS, atomic commit, total order broadcast, locks/leases, uniqueness constraints, and leader election are all *reducible to consensus*. Solve one, you've solved them all. This is why ZooKeeper sits under half the infrastructure you use.

3. Leader per epoch, not forever

Consensus uses a leader, but electing a leader needs consensus — not a contradiction. Escape via epoch numbers: you don't need a unique leader forever, only a unique leader *per epoch*.

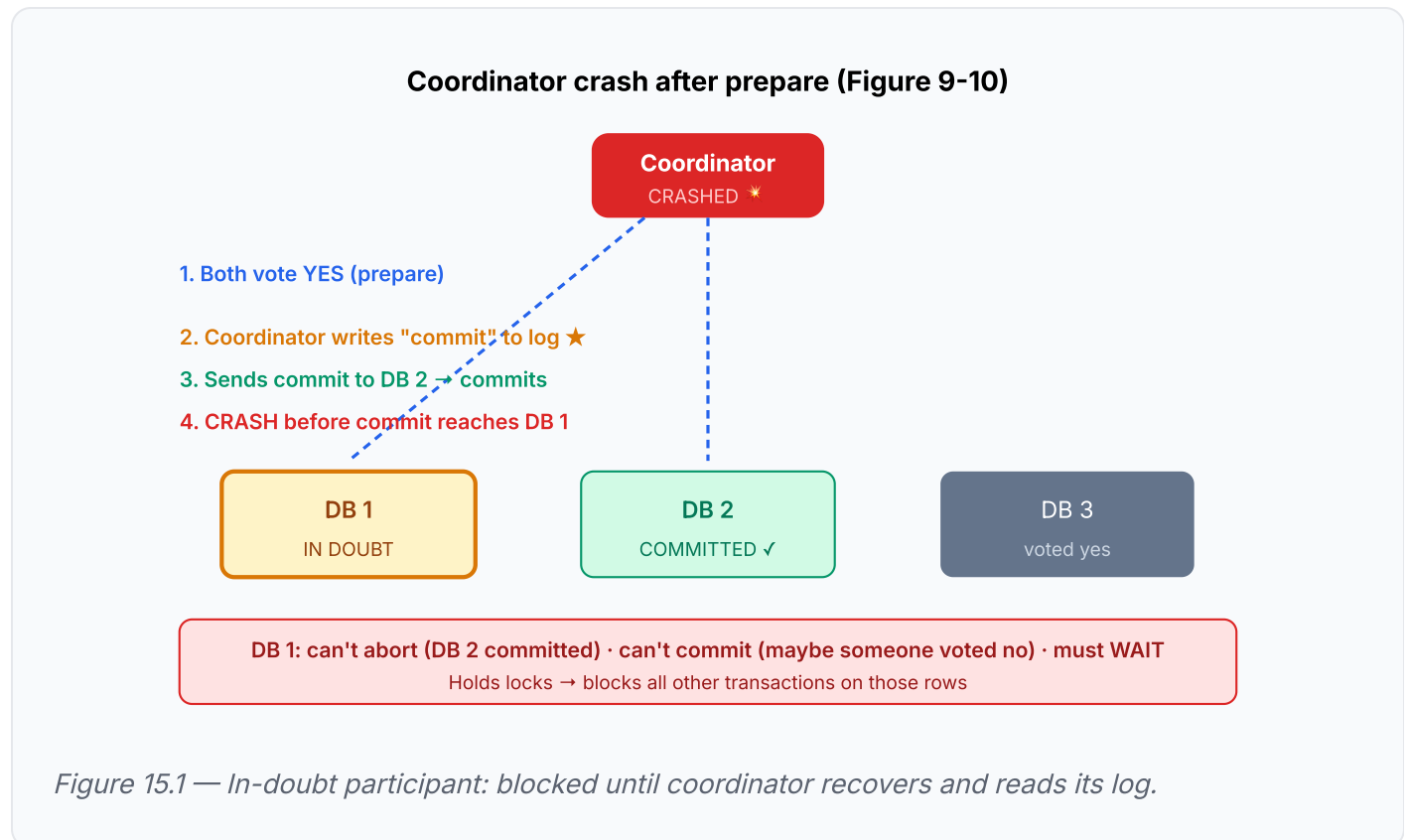
14. Anti-Patterns

Anti-pattern	Why it fails
Reasoning with "CP vs AP" labels	Most real systems fit neither cleanly; use specific guarantees instead
Assuming strict quorums ($w+r>n$) give linearizability	They don't — concurrent readers see old and new values; no linearizable CAS without consensus
Using Lamport clocks for uniqueness constraints	Winner only known after the fact; concurrent claims both "succeed" temporarily
Lock without fencing tokens	Paused (GC) old leader wakes and corrupts data — lock said it was unique, but it wasn't
Leaving XA in-doubt transactions to rot	Orphaned transactions hold row locks forever until admin resolves; heuristic decisions break atomicity
Treating 2PC as fault-tolerant consensus	2PC has safety, not termination — coordinator crash blocks everything
Implementing consensus yourself	Raft looks simple; edge cases (membership changes, snapshots) are treacherous. Use etcd/ZK.

15. Worked Example: 2PC Coordinator Crash

Setup

App completes writes on databases 1 and 2, then the coordinator runs phase 1: *prepare* to both. Both vote "yes" — each has durably written the transaction and irrevocably promised to commit if told to.



Why DB 1 is stuck

- ✗ **Can't unilaterally abort** — DB 2 may have committed; unilateral abort breaks atomicity
- ✗ **Can't unilaterally commit** — maybe another participant voted no
- ✗ **Can't ask DB 2 safely** — doesn't know the coordinator's decision
- ✓ **Only way forward:** wait for coordinator to recover and read its decision from the transaction log — the commit point of 2PC is really a single-node atomic commit on the coordinator

Meanwhile: the in-doubt participant holds its locks, blocking every other transaction touching those rows. This is why 2PC is a *blocking* protocol. 3PC "fixes" it only by assuming bounded delays and a perfect failure detector — which real networks don't provide.

Bonus: Linearizability violation

Alice and Bob check the 2014 World Cup final score. Alice refreshes, sees the result, exclaims it. Bob then refreshes and gets a stale "game in progress" from a lagging replica. Because Bob's query *started after* Alice's completed (via a side channel — her voice), returning the older value violates linearizability's recency guarantee.

16. Problems Each Mechanism Solves

Mechanism	Problem it solves	Example
Total order broadcast via sequencer	Matching engine must be deterministic and replayable — every replica applies operations in identical order	Stock exchange
Raft/Paxos consensus (not heartbeat failover)	Eliminate split-brain entirely, not just reduce its odds, on systems holding money/placement state	S3-like storage placement, digital wallet, stock exchange failover
ZooKeeper/etcd for coordination	Many independent servers need one agreed-on assignment (which server owns which channel/shard/target)	Chat system (server assignment), nearby friends (pub/sub channel ring), distributed MQ (broker metadata + leader election), metrics monitoring
Two-phase commit for cross-service atomicity	Multiple services must commit or abort together despite no shared DB transaction	Hotel reservation (2PC vs Saga), digital wallet
CAP / CP-vs-AP framing	Forces a stated choice between strict consistency and availability during a partition	Leaderless key-value store
Per-partition local order vs. global order	Global total order is overkill when only per-entity order matters	Chat system (per-channel sequence number over global Snowflake ID), unique-ID generator

17. Connects To (Cross-Chapter)

Chapter	Connection	Interview soundbite
Ch 5 Replication	Single-leader logs $\approx$ total order broadcast; leaderless quorums and LWW explain why Dynamo-style systems aren't linearizable.	"Quorum overlap $\neq$ linearizability. Need consensus for single-copy semantics."
Ch 7 Transactions	Serializability, 2PL, SSI — the isolation half of strict serializability; single-node atomic commit underlies 2PC.	"SSI is serializable but not linearizable — snapshot reads are stale by design."
Ch 8 Distributed Troubles	Process pauses, unbounded delays, and clock skew are exactly why fencing tokens and epochs exist.	"A lock without a fencing token is a suggestion, not a guarantee."
Ch 11 Stream Processing	Total order broadcast as a log foreshadows event logs; exactly-once semantics returns there.	"Consensus log $\rightarrow$ event stream $\rightarrow$ exactly-once processing."

18. Key Takeaways

1. **Linearizability = recency on a single object**; serializability = serial-order illusion for multi-object transactions. Know the distinction cold; combined they're strict serializability.
2. **Wide swaths of coordination** — leader election, atomic commit, locks, uniqueness, total order broadcast, linearizable CAS — are all equivalent to consensus.
3. **2PC gives atomic commit but blocks** on coordinator failure; fault-tolerant consensus (Paxos/Raft/Zab/VSR) adds termination via majority quorums and epochs.
4. **Causal consistency is the practical middle ground**: preserves what users perceive as "order" without linearizability's latency and availability cost.
5. **FLP says consensus is impossible** in the pure asynchronous model — timeouts and randomness rescue it in practice.
6. **Don't implement consensus yourself**: use ZooKeeper/etcd, and use fencing tokens with any lock or lease.
7. **If your system has a leader and can't tolerate its loss**, you either wait, pick manually (failover), or run a consensus algorithm — and consensus needs a majority alive.

19. Junior SWE Checkpoints

Q1: What's the difference between linearizability and serializability?

Linearizability = recency guarantee on a *single object* (register). Serializability = isolation property on *multi-object transactions* (some serial order). Both together = strict serializability. SSI is serializable but NOT linearizable.

Q2: Does $w+r>n$ quorum replication give linearizability?

No. Concurrent readers can still see old and new values. Overlap guarantees freshness for *completed* writes, not a single-copy illusion. Need consensus (Raft/ZK) for linearizability.

Q3: 2PC coordinator crashes after all participants vote yes but before sending commit to DB 1. What happens?

DB 1 is **in doubt** — can't abort (DB 2 may have committed), can't commit (maybe someone voted no). Holds locks, blocks other transactions. Must wait for coordinator recovery. This is why 2PC is blocking and not fault-tolerant consensus.

Q4: Why can't you use Lamport timestamps for username uniqueness?

Lamport gives a total order but can't detect concurrency. Two clients can both "claim" a username with concurrent timestamps — winner only known after collecting all operations. Use consensus CAS on a log instead.

Q5: What problem does FLP prove, and why doesn't it stop us?

Consensus impossible with deterministic algorithms in pure async model (one crash). We cheat with timeouts (partial synchrony) and randomness (Raft election timeouts). Great theory, limited practical impact.

Q6: Why do you need fencing tokens with distributed locks?

A paused (GC) old leader can wake after a new leader is elected. Without fencing, both write — data corruption. Fencing token (monotonic zxid) lets downstream services reject stale leader's requests.

Q7: What's equivalent to consensus?

Linearizable CAS, atomic commit, total order broadcast, locks/leases, uniqueness constraints, leader election — all reducible to each other. Solve one, solve them all. This is why ZooKeeper/etcd underpin so much infrastructure.

Glossary

Term	Definition
Linearizability	Single-object recency guarantee — single-copy illusion
Serializability	Multi-object transaction isolation — some serial order
Strict serializability	Both linearizability and serializability combined
Two-phase commit (2PC)	Blocking atomic commit protocol with coordinator
Consensus	Agreement with termination — Raft, Paxos, Zab, VSR
Total order broadcast	Reliable delivery in same order to all nodes
Causal consistency	Preserves happens-before without total order
Lamport timestamp	(counter, nodeID) — total order consistent with causality
Version vector	Per-replica counters — detects concurrency
Epoch / term / ballot	Monotonic leader generation number
Fencing token	Monotonic ID (zxid) rejecting stale leader requests
FLP impossibility	Consensus impossible in pure async with one crash
In-doubt transaction	2PC participant waiting for coordinator decision